Programação Orientada a Objetos

Departamento de Computação
Universidade Federal de Sergipe

Classes Essenciais da API Java

Prof. Kalil Araujo Bispo
kalil@dcomp.ufs.br

Roteiro

- Java API
- Pacote java.lang
- Pacote java.math
- Pacote java.util

Java API

- **O que é a Java API?**
  - Java *Application Programming Interface*
  - Centenas de classes predefinidas e compiladas (bibliotecas)
  - Composta por 2 tipos de recursos distintos: classes e interfaces
    - Centenas de Interfaces e milhares de classes que acompanham o kit de desenvolvimento do Java
    - Podem ser empregadas para a realização de diversos tipos de tarefas durante a construção de um programa

Java API

- **Para usar uma classe da API**
  - É preciso descobrir em qual pacote ela está
    - Que classes existem na biblioteca?
    - Como descobrir o que cada uma dela faz?

Java API

Overview (Java Plat

https://docs.oracle.com/javase/7/docs/api/

Java™ Platform Standard Ed. 7

All Classes

Packages

java.applet
java.awt
java.awt.color
java.awt.datatransfer
java.awt.dnd
java.awt.event
java.awt.font
java.awt.geom
java.awt.im
java.awt.im.spi
java.awt.image
java.awt.image.renderable
java.awt.print

All Classes

AbstractAction
AbstractAnnotationValueVisitor6
AbstractAnnotationValueVisitor7
AbstractBorder
AbstractButton
AbstractCellEditor
AbstractCollection
AbstractColorChooserPanel
AbstractDocument
AbstractDocument.AttributeContext
AbstractDocument.Content
AbstractDocument.ElementEdit
AbstractElementVisitor6
AbstractElementVisitor7
AbstractExecutorService
AbstractInterruptibleChannel
AbstractLayoutCache
AbstractLayoutCache.NodeDimensions
AbstractList
AbstractListModel
AbstractMap
AbstractMap.SimpleEntry
AbstractMap.SimpleImmutableEntry
AbstractMarshallerImpl
AbstractMethodError
AbstractOwnableSynchronizer
AbstractPreferences
AbstractProcessor
AbstractQueue
AbstractQueuedLongSynchronizer
AbstractQueuedSynchronizer
AbstractRegionPainter
AbstractRegionPainter.PaintContext
AbstractRegionPainter.PaintContext.CacheMode
AbstractScriptEngine
AbstractSelectableChannel
AbstractSelectionKey
AbstractSelector
AbstractSequentialList
AbstractSet
AbstractSpinnerModel
AbstractTableModel
AbstractTypeVisitor6
AbstractTypeVisitor7
AbstractUndoableEdit
AbstractUnmarshallerImpl
AbstractWriter

Overview Package Class Use Tree Deprecated Index Help

Java™ Platform Standard Ed. 7

Prev Next Frames No Frames

Java™ Platform, Standard Edition 7
API Specification

This document is the API specification for the Java™ Platform, Standard Edition.

See: Description

Packages

Package	Description
java.applet	Provides the classes necessary to create an applet and the classes an applet uses to communicate with its applet context.
java.awt	Contains all of the classes for creating user interfaces and for painting graphics and images.
java.awt.color	Provides classes for color spaces.
java.awt.datatransfer	Provides interfaces and classes for transferring data between and within applications.
java.awt.dnd	Drag and Drop is a direct manipulation gesture found in many Graphical User Interface systems that provides a mechanism to transfer information between two entities logically associated with presentation elements in the GUI.
java.awt.event	Provides interfaces and classes for dealing with different types of events fired by AWT components.
java.awt.font	Provides classes and interface relating to fonts.
java.awt.geom	Provides the Java 2D classes for defining and performing operations on objects related to two-dimensional geometry.
java.awt.im	Provides classes and interfaces for the input method framework.
java.awt.im.spi	Provides interfaces that enable the development of input methods that can be used with any Java runtime environment.
java.awt.image	Provides classes for creating and modifying images.
java.awt.image.renderable	Provides classes and interfaces for producing rendering-independent images.
java.awt.print	Provides classes and interfaces for a general printing API.
java.beans	Contains classes related to developing *beans* -- components based on the JavaBeans™ architecture.
java.beans.beancontext	Provides classes and interfaces relating to bean context.
java.io	Provides for system input and output through data streams, serialization and the file system.
java.lang	Provides classes that are fundamental to the design of the Java programming language.
java.lang.annotation	Provides library support for the Java programming language annotation facility.
java.lang.instrument	Provides services that allow Java programming language agents to instrument programs running on the JVM.
java.lang.invoke	The `java.lang.invoke` package contains dynamic language support provided directly by the Java core class libraries and virtual machine.
java.lang.management	Provides the management interfaces for monitoring and management of the Java virtual machine and other components in the Java runtime.
java.lang.ref	Provides reference-object classes, which support a limited degree of interaction with the garbage collector.
java.lang.reflect	Provides classes and interfaces for obtaining reflective information about classes and objects.
java.math	Provides classes for performing arbitrary-precision integer arithmetic (`BigInteger`) and arbitrary-precision decimal arithmetic (`BigDecimal`).
java.net	Provides the classes for implementing networking applications.
java.nio	Defines buffers, which are containers for data, and provides an overview of the other NIO packages.
java.nio.channels	Defines channels, which represent connections to entities that are capable of performing I/O operations, such as files and sockets; defines selectors, for multiplexed, non-blocking I/O operations.
java.nio.channels.spi	Service-provider classes for the `java.nio.channels` package.
java.nio.charset	Defines charsets, decoders, and encoders, for translating between bytes and Unicode characters.
java.nio.charset.spi	Service-provider classes for the `java.nio.charset` package.
java.nio.file	Defines interfaces and classes for the Java virtual machine to access files, file attributes, and file systems.

Java API

- **A documentação da API não vem incorporada ao download do jdk**
  - http://download.oracle.com/javase/

Java API

- **Quais APIs existem?**
  - Java SE (padrão)
  - Java EE
  - Java ME
  - Java DB
  - Java FX
  - ...

Pacote java.lang

- **Provê classes fundamentais para a programação Java**
  - Object – raiz da hierarquia de classes
  - Class – permite identificar instâncias em tempo de execução
  - Package – permite a implementação de pacotes

- **Único pacote que não precisa ser importado**

Pacote java.lang

- **Entendendo melhor classes que já estamos manipulando...**
  - **System**: rotinas do sistema
  - **String**: manipulação de cadeias de caracteres.

Pacote java.lang

- **Classe System**
  - Não pode ser instanciada
  - Provê variáveis estáticas que representam a entrada padrão, saída padrão, saída de erros e outros

System
«final» InputStream in «final» PrintStream out «final» PrintStream err ...

Pacote java.lang

- **Classe System**
  - Métodos
    - **currentTimeMillis**: retorna o tempo corrente em milisegundos

```
long horaSistema = System.currentTimeMillis();
```

    - **exit**: encerra a JVM
    - **gc**: executa o coletor de lixo

Pacote java.lang

- **Classe System**
  - Métodos (continuação)
    - **getProperties**: Determina as propriedades atuais do sistema
    - **setIn**: Altera a Stream de entrada padrão
    - **setOut**: Altera a Stream de saída padrão
    - **setErr**: Altera a Stream de saída de erro padrão

Pacote java.lang

- **Anatomia do System.out.println(....)**

out é uma variável estática de System

System
«final» InputStream in
«final» **PrintStream out**
«final» PrintStream err
...

out referencia a um objeto que representa a saída padrão

println é um método de PrintStream

Pacote java.lang

- **String**
  - Não é um tipo primitivo. É um objeto!
  - Strings em Java são instâncias da classe java.lang.String

```
String dog = “Braddock”;
```

  - Armazena uma cadeia de caracteres

Pacote java.lang

- **Como saber se duas strings são iguais?**

```
String dog1 = “Braddock”;
String dog2 = “Rambo”;

if ( dog1 == dog2 )
   System.out.println(“São iguais”);
else
  System.out.printlm(“São diferentes”);
```

Pacote java.lang

- **Comparação através do método equals**

```
String dog1 = “Braddock”;
String dog2 = “Rambo”;

if ( dog1.equals(dog2) )
   System.out.println(“São iguais”);
else
   System.out.printlm(“São diferentes”);
```

A comparação ocorre de char em char

Pacote java.lang

- **String - principais métodos:**
  - **Equals**
    - Retorna true quando a String passada como argumento é igual à String cujo método foi chamado
  - **Length**
    - Retorna o comprimento da String

Pacote java.lang

- **String - principais métodos:**
  - **Substring**
    - Retorna uma nova String que representa uma parte da String cujo método foi chamado

```
String texto = “Braddock e Rambo são meus animais de estimação”;
  String dog2 = texto.substring(11, 15);
```

Pacote java.lang

- **String - principais métodos:**
  - **CharAt**
    - Retorna o caractere localizado na posição especificada (índices variam de 0 a length() – 1)
  - **IndexOf**
    - Retorna a posição do primeiro caractere que coincide com o passado como argumento

Pacote java.lang

- **String - principais métodos:**
  - **ToLowerCase**
    - Retorna uma String com os caracteres minúsculos
  - **toUpperCase**
    - Retorna uma String com os caracteres maiúsculos

```
String nome = “Arnold Schwarzenegger”;
String nomeUpper = nome.toUpperCase();
```

Pacote java.lang

- **String**
  - Uma String é imutável

```
String nome = “Arnold Schwarzenegger”;
nome.toUpperCase();
System.out.println(nome);
```

  - Como será impresso?

Pacote java.lang

- **E agora, como será impresso?**

```
String nome = “Arnold Schwarzenegger”;
nome = nome.toUpperCase();
System.out.println(nome);
```

- **Criação de Strings temporárias**
  - Perda de desempenho

Pacote java.lang

- **Classes importantes ainda não usadas**
  - StringBuilder
  - StringBuffer
  - Math
  - Classes Wrappers

Pacote java.lang

- **StringBuilder e StringBuffer**

StringBuilder
StringBuilder( ) StringBuilder(int) StringBuilder(String) StringBuilder append*(primitive)* StringBuilder append(Object) StringBuilder insert(int,*primitive*) StringBuilder insert(int,Object) char charAt(int) StringBuilder delete(int,int) String toString() ....

Strings que podem ser alterados dinamicamente

StringBuffer tem a mesma funcionalidade que StringBuilder, porém serve para contextos onde há concorrência

Pacote java.lang

- **Exemplo para inverter uma string**
  - Usando String → implementação ineficiente

```
public static String reverseStr(String source) {
    String resp = "";
    for (int i = 0; i < source.length() ; i++)
        resp = source.charAt(i) + resp;
    return resp;
}
```

Cada concatenação cria um String novo
O "custo" da operação é alto

Pacote java.lang

- **Exemplo para inverter uma string**
  - Usando StringBuilder

```
public static String reverseStr(String source) {
    int len = source.length();
    StringBuilder dest = new StringBuilder(len);

    for (int i = (len - 1); i >= 0; i--)
        dest.append(source.charAt(i));
    return dest.toString();
}
```

Obs. StringBuilder já tem um método reverse

Pacote java.lang

- **Math**
  - Possibilitar a execução de operações matemáticas
  - Constantes
    - **Math.PI** = 3,14...
    - **Math.E** = base de logaritmos
  - Métodos Estáticos
    - **Math.min(x,y)**: menor entre 2 valores
    - **Math.max(x,y)**: maior entre 2 valores
    - **Math.random()**: gera número aleatório >= 0 e < 1

Pacote java.lang

- **Math (outros métodos)**
  - **Math.pow(x,y)**: x elevado a y
  - **Math.round(x)**: arredonda um número real
  - **Math.sqrt(x)**: raiz quadrada
  - **Math.tan(x)**: Calcula a tangente
  - **Math.sin(x)**: Calcula o seno
  - **Math.cos(x)**: Calcula o coseno
  - **Math.exp(x)**: E elevado a X
  - **Math.log(x)**: Logaritmo na base E

Pacote java.lang

- **Exemplo**

```
public static void main(String[] args) {
    int r = 2;
    double areaCirculo = Math.PI * Math.pow(r, 2);

    System.out.println(areaCirculo);

    double d = 4.5;
    long i = Math.round(d);
    System.out.println(i);

    int x = -4;
    int y = Math.abs(x);
    System.out.println(y);
}
```

Pacote java.lang

- **Classes Wrappers**
  - Empacotam valores contidos em variáveis de tipos primitivos em objetos
  - Objetivo
    - Oferecer funcionalidades e facilidades para a manipulação desses tipos

Pacote java.lang

- **Classe Boolean (wrapper)**
  - Empacota o tipo primitivo boolean
  - Provê métodos de conversão
  - Exemplos
    - **toString(boolean b)**: retorna um objeto String representando o valor
    - **valueOf(String s)**: retorna um objeto Boolean a partir do conteúdo da string

Pacote java.lang

- **Classe Character (wrapper)**
  - Empacota o tipo primitivo char
  - Provê métodos de categorização e conversão
  - Exemplos
    - **isDigit(charc c)**: Retorna true quando o caractere é um dígito
    - **isLetter(charc c)**: Retorna true quando o caractere é uma letra
    - **isLetterOrDigit(charc c)**: Retorna true quando o caractere é um dígito ou uma letra

Pacote java.lang

- **Classe Character (wrapper)**
  - Exemplos
    - **isLowerCase(charc c)**: Retorna true se o caractere for minúsculo
    - **isUpperCase(charc c)**: Retorna true se o caractere for maiúsculo
    - **toLowerCase(charc c)**: Retorna o caractere minúsculo correspondente
    - **toUpperCase(charc c)**: Retorna o caractere maiúsculo correspondente

Pacote java.lang

- **Classes Wrappers Numéricas**
  - Short, Byte, Integer, Long, Float e Double
  - Empacotam o tipo primitivo respectivo
  - Provê métodos de conversão e manipulação

  - Exemplos
    - **valueOf (String s)**: cria uma instância da classe wrapper a partir do valor contido na String passada
    - **parseTipo (String s)**: retorna o valor contido na String em uma variável do tipo primitivo correspondente
    - **toString(tipoPrimitivo i)**: retorna um objeto String representando o tipo primitivo especificado

Pacote java.lang

- **Classes Wrapper numéricas**
  - Exemplos

```
String s1 = "14.5";
Float f1 =
Float.valueOf(s1);
```

```
String s2 = "101";
int i1 = Integer.parseInt(s2);
```

```
Float f2 = new Float(9.5f);
Int i2 = f2.intValue();
```

Pacote java.math

- **Pacote que provê classes numéricas com grande capacidade**
  - BigInteger: Inteiro
  - BigDecimal: Decimal
- **Seus objetos**
  - São imutáveis
  - Têm tamanho indeterminado
- **Operações que podem ser realizadas**
  - adição, subtração, multiplicação, divisão...
  - Conversão para tipos primitivos

Pacote java.util

- **Provê uma miscelânea de classes utilitárias**
  - Tratamento de datas e tempo
  - Impressão e entrada de dados
  - Coleções
  - Internacionalização
  - ...

Pacote java.util

- **Tratamento de datas e tempo**
  - Classe Date
  - Classe Calendar

Pacote java.util

- **Classe Date**
  - Representa um instante específico no tempo, com precisão de milissegundos
  - Construtores
    - **Date()**
      - Cria o objeto contendo a data/hora atual
    - **Date (long l)**
      - Recebe um número long que representa o número de milisegundos a partir de 1º de Janeiro de 1970, 0h GMT.

Pacote java.util

- **Classe Date**
  - Métodos
    - **Comparação com outro Date**
      - after, before, compareTo e equals
    - **getTime**: retorna o número em milisegundos armazenado no objeto
    - **setTime**: altera o número em milisegundos armazenado no objeto
    - **clone**: clona o objeto
    - **toString**: converte a data em String

Pacote java.util

- **Classe Calendar**
  - Possibilita o tratamento de diversos tipos de calendários
  - Atributos
    - Identificam componentes de uma data
    - YEAR
    - MONTH
    - DAY_OF_MONTH
    - DAY_OF_WEEK
    - DAY_OF_YEAR
    - HOUR_OF_DAY

Pacote java.util

- **Classe Calendar**
  - Métodos
    - **getInstance**: Retorna o calendário correspondente ao fuso horário e ao local onde está sendo executado
    - **getTime**: Retorna a data armazenada no calendário
    - **getTimeInMillis**: Retorna a data armazenada no calendário em milisegundos
    - **setTime**: Altera a data armazenada no calendário
    - **setTimeInMillis**: Altera a data armazenada no calendário passando um novo valor em milisegundos

Pacote java.util

- **Classe Calendar**
  - Métodos
    - **add**: Adiciona a um campo da data um valor
    - **Comparação**: métodos after, before e equals
    - **clone**: Clona o objeto
    - **get**: Recebe como parâmetro uma das constantes definas nessa classe que identificam os componentes da data e retorna seu valor
    - **set**: Altera o valor de um ou mais campos
    - **roll**: adiciona ou subtrai o valor de um campo

Referências

- **Slides “Essenciais” Prof Giovanny . Java.UFS. 2009.**
- **Slides “Classes Básicas Utilitárias” e “Tipos Estruturados”. Prof Alberto Costa Neto. Java. UFS. 2004.**
- **Caelum. Java e Orientação a Objetos**
  - http://www.caelum.com.br/apostilas/
  - Capítulos 14 e 15

Programação Orientada a Objetos

Departamento de Computação
Universidade Federal de Sergipe

Classes Essenciais da API Java

Prof. Kalil Araujo Bispo
kalil@dcomp.ufs.br